HR Portal Multi-language Change Report

Oct 15, 2026 · @Diamond Insurance Broker IT

The HR portal now works in English (default) and Arabic (right-to-left). This was done by moving hard-coded text into language files, not by changing how features behave. The database gets **one new migration that only adds 8 columns**. No table is created, dropped or renamed, and no existing column or data is changed. All work is uncommitted on branch `languageconversion`. It touches 236 tracked files (131 backend, 105 frontend) and adds about 70 new files, most of them language files.

There are two separate language settings. **Screen language** is what a user sees in the portal. **Communication language** is the language of every email sent to an employee. An employee can use the English portal and still get Arabic emails.

Database changes

Yes, there are database changes: 8 new columns, all added by one migration, `backend/database/migrations/2026_10_15_000001_add_multilanguage_support.php`. Every column is new and either has a default or is nullable, so existing rows and existing code keep working unchanged. The migration checks before each step (`hasColumn` / `hasTable`), so it is safe to run twice.

Table	New column	Type	Default	Placed after	Purpose
`users`	`preferred_language`	VARCHAR(5)	`'en'`	`email`	Screen language, remembered across devices
`employees`	`preferred_communication_language`	VARCHAR(5)	`'en'`	`email`	Language of every email sent to the employee
`departments`	`name_ar`	VARCHAR(150), nullable	NULL	`name`	Arabic department name
`designations`	`title_ar`	VARCHAR(150), nullable	NULL	`title`	Arabic job title
`leave_types`	`name_ar`	VARCHAR(150), nullable	NULL	`name`	Arabic leave type name
`loan_types`	`name_ar`	VARCHAR(150), nullable	NULL	`name`	Arabic loan type name
`request_types`	`name_ar`	VARCHAR(150), nullable	NULL	`name`	Arabic request type name
`units`	`name_ar`	VARCHAR(150), nullable	NULL	`name`	Arabic unit name

- **Existing data:** nothing is changed. Every employee starts on English emails and every user on the English screen. An empty Arabic name falls back to the English one.
- **Not changed:** status values stay as English codes in the database (`pending` , `manager_approved` and so on). They are translated on screen and in emails from language files, never stored translated. Employees already had `arabic_name` , which is reused.
- **Rollback:** `php artisan migrate:rollback` drops the 8 columns. Any Arabic names or language choices entered by then are lost.
- **Status on this machine:** the migration has not been run here. The dev database has no `migrations` table yet.

Backend changes (Laravel)

Most backend edits wrap existing English text in `__()` so Laravel can return it in Arabic. Business rules, queries and permissions are unchanged, except for the email sending split described in the next section. 131 existing backend files were edited.

Area	Files	What changed in existing code
Request pipeline	`Http/Kernel.php`	New `SetLocale` middleware added to the `api` group. It reads the SPA's `Accept-Language` header so validation errors and API messages come back in the user's language.
API controllers	30 in `Controllers/API` , plus 3 others	Response messages wrapped in `__()` , e.g. `'Leave submitted'` → `__(':type of :hoursh submitted', …)` . Admin master-data screens accept and return the new `name_ar` / `title_ar` fields.
New endpoints	`routes/api.php` , `AuthController` , `ProfileController`	`PUT /api/v1/auth/language` saves the screen language. `PUT /api/v1/profile/communication-language` saves the email language. Login, `/auth/me` and `GET /profile` now also return both settings.
Models	`User` , `Employee` , `Department` , `Designation` , `LeaveType` , `LoanType` , `RequestType` , `Unit`	New columns added to `$fillable` . `User` now implements `HasLocalePreference` , so Laravel notifications such as password reset use the employee's email language.

Area	Files	What changed in existing code
Validation and resources	`StoreEmployeeRequest`, `UpdateEmployeeRequest`, `LoginRequest`, `EmployeeResource`, `ContractResource`	HR can set `preferred_communication_language` (`en` or `ar` only). Resources also return the Arabic names. Login messages translated.
Mail classes	35 in `app/Mail`	Each now extends the new `BaseMailable` (the 4 leave mails do so through `LeaveStatusMail`). Subjects use `__()`. `BaseMailable` picks the language from the recipient's `preferred_communication_language`.
Email templates	22 in `resources/views/emails`	Hard-coded English replaced by `__()` keys. Templates set `lang` and `dir` and align text right for Arabic. A new reset-password template was added.
Services	20 in `app/Services`	Messages and notification text translated. Email sending changed in the 7 services covered in the next section.

New backend files

- `app/Http/Middleware/SetLocale.php`: applies the request language.
- `app/Mail/BaseMailable.php` and `app/Mail/Support/MailFormat.php`: base class for all emails, plus date and number formatting.
- `app/Support/Locale/`: `SupportedLocales` (en, ar), `RecipientLocale` (email address → language), `StatusLabel` (status code → label), `LocalName` (Arabic name with English fallback), `LocalizedMail` (per-language sending).
- `lang/ar.json`: Arabic versions of API messages.
- `lang/en` and `lang/ar`: `mail.php`, 13 `mail_<module>.php` files and a generated `status.php`. Arabic also gets `auth`, `pagination`, `passwords` and `validation`.

Email sending: one message per language

Some emails used to go out as one message to several people. One message can only be in one language, and it followed the first "To" person. These emails are now sent as one message per language group. Content, recipients and attachments are unchanged.

Email	File	Before	Now
Contract renewal reminder	`ContractRenewalNotificationService`	1 message: manager in To, HR in CC	1 per language: manager keeps To, HR keeps CC
Employee document uploaded	`EmployeeController`, `PublicOnboardingController`	1 message: first HR in To, rest in CC	1 per language: first HR of each group in To, rest in CC
Onboarding tasks assigned	`OnboardingTaskNotificationService`	1 message to the whole list	1 per language
New hire IT notice	`NewHireNotificationService`	1 message to the list	1 per language
New job / application created	`RecruitmentService`	1 message to all HR managers	1 per language
Interview invite	`RecruitmentService`	1 message: applicant + interviewers in To, test address in CC	Applicant gets their own English copy (test address in CC). Interviewers get copies in their own languages.
Offer letter	`RecruitmentService`	1 message: applicant in To, HR managers + test address in CC	Applicant gets their own English copy (test address in CC). HR managers get copies in their own languages.
Annual leave contract expiry	`AnnualLeaveContractExpiryNotificationService`	1 message: employee in To, manager + HR in CC	1 per language: employee keeps To, others keep CC
Missed checkout	`MissedCheckoutNotificationService`	1 message: employee in To, HR in CC	1 per language: employee keeps To, HR keeps CC

- **How it works:** all of these flows use one helper, `App\Support\Locale\LocalizedMail::send($to, $cc, …)`. Within a language group people keep their To or CC role. If a group has only CC people, the first of them moves

to To.

- **What recipients notice:** people no longer see colleagues who are in a different language group. Applicants no longer see the interviewers' or HR managers' addresses.
- **Partial sends:** if one group's message fails, groups sent before it have already gone out. Before, it was all or nothing.
- **Not changed:** the birthday wish email, which is bilingual content written by HR and sent to all staff. The new-employee announcement already sent one message per language.

Local test-email redirect

`AppServiceProvider` now sends every email, including CC and BCC, only to `MAIL_TEST_EMAIL` when `APP_ENV=local`. Before, this happened only with `MAIL_FORCE_TEST_EMAIL=true`. On this machine `.env` was changed from `APP_ENV=dev` to `APP_ENV=local`, and `MAIL_FORCE_TEST_EMAIL=true` was added. `.env` is not in git, so this change stays on this machine. Production must not use `APP_ENV=local`.

Frontend changes (Angular)

Frontend edits replace on-screen English text with translation keys and add right-to-left layout for Arabic. 105 existing files were edited, 94 of them in the feature modules. Screens, routes and API calls work as before. No new npm package was added: `@ngx-translate/core` and `@ngx-translate/http-loader` were already in `package.json`.

Area	Files	What changed in existing code
App setup	`app.module.ts`, `app.component.ts`, `index.html`	Translation module and RTL support registered. A new interceptor sends `Accept-Language` with every API call. The language loads before the app starts. The root element switches `dir` between `ltr` and `rtl`. The Arabic font (IBM Plex Sans Arabic) is loaded.
Login and session	`auth.service.ts`, `auth.effects.ts`	After login and on `/auth/me`, the user's saved screen language is applied.

Area	Files	What changed in existing code
Main layout	`main-layout` component (4 files)	Menu labels became keys (`'Dashboard'` → `'NAV.DASHBOARD'`). An EN / عربي switcher was added to the sidebar and the mobile top bar. Mobile drawer layout fixed.
Feature screens	94 files across 20 modules	Text in templates uses the `translate` pipe. Toasts and confirm messages in TypeScript use `LanguageService` . Status codes are shown with the `statusLabel` pipe. Arabic names are shown with `locName` . Dates use `locDate` for Arabic month names.
Employee form and My Profile	`employees` , `profile` modules	HR can set an employee's communication language. Employees can change their own on My Profile.
Admin master data	`admin` , `leave` , `loans` , `requests` modules	Arabic name fields for departments, job titles, units, leave types, loan types and request types.
Styles	`styles.scss` and component SCSS	Left/right spacing changed to logical properties so layouts mirror in Arabic. Shared rules added for tablet (⩽1024px) and mobile (⩽768px) sizes.

New frontend files

- `core/services/language.service.ts` : current language, direction and the `t()` helper.
- `core/i18n/i18n.config.ts` and `core/i18n/multi-file-translate.loader.ts` : the list of translation files and their loader.
- `core/interceptors/language.interceptor.ts` : adds the `Accept-Language` header.
- `shared/components/language-switcher` : the EN / عربي switcher.
- `shared/pipes/i18n.pipes.ts` : the `statusLabel` , `locName` and `locDate` pipes.

- `assets/i18n/en` and `assets/i18n/ar`: 20 translation files each, one per module plus `common.json`.

Deployment, testing and open items

Deployment steps

1. Back up the database.
2. Deploy the backend and run `php artisan migrate`. This adds the 8 columns.
3. Run `php artisan config:clear` and `php artisan view:clear` so the new language files and templates are picked up.
4. Build and deploy the frontend (`ng build`). The `assets/i18n` files must be included.
5. Check the server `.env`: `APP_ENV` must not be `local`, or every email goes to the test address. Set `MAIL_FORCE_TEST_EMAIL=false` on production.
6. Optional: HR enters the Arabic names for departments, job titles, units, leave types, loan types and request types. Until then the English names are shown.

Testing done

- New test `tests/Feature/PerLanguageNotificationMailTest.php`: 9 tests, all passing. For each email in the table above, it checks the number of messages, the To and CC people, and the language of each one. No real email is sent.
- All changed PHP files pass the PHP syntax check (`php -l`).
- Not yet done: a full manual run through every screen in Arabic, and checking real emails in an inbox.

Open items and risks

- [ ] Commit the work. Everything is still uncommitted on `languageconversion`.
- [ ] Run the migration on the dev database. Its `migrations` table does not exist yet.
- [ ] Most existing automated tests cannot run, because the migrations fail on the SQLite test database. This problem was already there before this work.
- [ ] Have an Arabic speaker review the wording in `lang/ar` and `assets/i18n/ar`.
- [ ] Status labels for emails: after editing `common.json` → `STATUS`, run `python tools/i18n/sync_status_labels.py` to regenerate `lang/*/status.php`.

The full developer guide is in `docs/MULTILANGUAGE.md` in the repository.